

SMART RECRUITMENT MATCHING PLATFORM

Full Technical Stack Document

ASP.NET Core Web API | SQL Server | Rule-Based Matching | Responsive Web UI

Prepared for Final Project Submission

Technical baseline verified from the final main branch

Document Date: 11 August 2026

Implementation note

The platform uses a deterministic, rule-based recruitment matching engine. It does not use external AI or machine-learning APIs for candidate scoring.

Document Purpose

This document provides a detailed description of the technologies, frameworks, languages, APIs, services, libraries and development tools used to build the Smart Recruitment Matching Platform. It is intended to satisfy the project submission requirement for a complete technical stack document and to provide a clear technical reference for assessors, developers and future maintainers.

Scope of This Document

- .NET version and runtime baseline
- Frontend technologies and user-interface implementation
- Backend framework, architecture and service organization
- Database platform and data-access technology
- Programming languages used across the solution
- Internal APIs, platform services and API documentation
- Security, authentication and authorization technologies
- Matching engine technologies and design
- Third-party libraries and NuGet packages
- Testing, development, source-control and tooling stack

Contents

1. Project Technology Overview
2. .NET Version Used
3. Frontend Technology / Framework
4. Backend Technology / Framework
5. Database and Data Access
6. Programming Languages
7. APIs / Services Used
8. Other Tools, Libraries and Technologies
9. Security and Authentication Stack
10. Matching Engine Technology
11. Testing and Quality Assurance Stack
12. Development, Version Control and Deployment Support
13. Final Technology Version Matrix
14. Conclusion

1. Project Technology Overview

The Smart Recruitment Matching Platform is implemented as a secure web application that connects Job Seekers, Employers and Administrators. The system combines a browser-based frontend with an ASP.NET Core Web API backend, Entity Framework Core data access, SQL Server LocalDB storage and JWT-based authentication. Candidate matching is performed by an internal deterministic scoring engine using skills, experience, education and location.

Core architectural style

Browser UI -> RESTful ASP.NET Core Web API -> Service Layer -> Repository Layer -> Entity Framework Core -> SQL Server LocalDB

Category	Technology / Version	Use in the Project
Platform	.NET 8 / SDK 8.0.300	Application runtime, build platform and common framework baseline.
Frontend	HTML5, CSS3, Vanilla JavaScript	Responsive pages, forms, dashboards, API calls, theme switching and user interactions.
Backend	ASP.NET Core Web API	REST endpoints, controllers, middleware, dependency injection, authentication and business services.
Database	SQL Server LocalDB	Relational storage for users, profiles, vacancies, skills, applications, notifications and contact requests.
ORM	Entity Framework Core 8.0.29	Object-relational mapping, DbContext, migrations, relationships and database queries.
Security	JWT Bearer + ASP.NET Core Identity PasswordHasher	Authentication, token validation, role-based authorization and secure password hashing.
API Documentation	Swagger / OpenAPI (Swashbuckle 6.6.2)	Interactive API documentation, request testing and Bearer-token authorization support.
Testing	xUnit, Moq, .NET Test SDK, coverlet	Automated unit tests, mocking, test execution and code-coverage collection support.

Category	Technology / Version	Use in the Project
Source Control	Git + GitHub	Branch-based team development, integration, history and final source distribution.

Architecture Characteristics

- Layered separation through Controllers, Services, Repositories, DTOs and Entities.
- Built-in ASP.NET Core dependency injection for registering repositories, services and matching components.
- RESTful JSON communication between the browser frontend and backend API.
- Role-aware access for Job Seeker, Employer and Administrator users.
- Database-backed matching and application workflows rather than hardcoded sample data.
- Static frontend files served directly by the ASP.NET Core application from wwwroot.

2. .NET Version Used

The project targets .NET 8.0. The application project uses the Microsoft.NET.Sdk.Web SDK and declares net8.0 as its TargetFramework. A global.json file standardizes the development SDK at version 8.0.300 with rollForward set to latestFeature, allowing compatible .NET 8 feature-band SDK updates while keeping the project on the .NET 8 major version.

Category	Technology / Version	Use in the Project
Target Framework	net8.0	Target framework for the Web API and test project.
SDK Baseline	8.0.300	Pinned through global.json for consistent command-line and IDE builds.
SDK Roll Forward	latestFeature	Allows compatible feature-band roll-forward within the selected .NET major/minor line.
Web SDK	Microsoft.NET.Sdk.Web	Enables ASP.NET Core web hosting, controllers, static files and web configuration.
Nullable References	Enabled	Improves null-safety analysis during C# compilation.
Implicit Usings	Enabled	Reduces repetitive framework using directives in .NET source files.

Verified build baseline

The finalized solution was validated on .NET 8 with a successful build reporting 0 warnings and 0 errors, and the automated test suite completed with 12 tests passed and 0 failed.

3. Frontend Technology / Framework

The frontend is implemented without a heavy client-side framework. It uses standards-based HTML5, CSS3 and Vanilla JavaScript, which keeps the application lightweight and allows it to be served directly by ASP.NET Core. The frontend communicates with the Web API through a shared JavaScript API client using the browser Fetch API.

3.1 HTML5

- Semantic page structure for the landing page, authentication pages and role-specific dashboards.
- Forms for registration, login, profiles, CV upload, vacancies and other user actions.
- Tables, cards, tabs and modal interfaces for applications, match details and administrative information.
- File input controls for CV upload using multipart FormData.

3.2 CSS3

- Shared responsive styling through `wwwroot/css/styles.css`.
- CSS variables for colors, backgrounds, text, borders, cards, controls and status indicators.
- Responsive layouts using grid and flexbox.
- Dark and Light theme support using shared theme variables.
- Theme persistence in `localStorage` and system-preference detection through `prefers-color-scheme`.
- Reusable button, badge, form, modal, table and card visual patterns.

3.3 JavaScript

- Vanilla JavaScript (ES6+) for DOM manipulation, events, asynchronous requests and page behavior.
- Fetch API for calling backend endpoints.
- Shared API client in `wwwroot/js/api.js` for request construction, Bearer tokens and error handling.
- `localStorage` for JWT/session-related client state and theme preference.
- `URLSearchParams` for vacancy search/filter query construction.
- `FormData` for CV uploads so binary files are not JSON-encoded.
- Role-based navigation and redirects after login.

3.4 Main Frontend Pages

Category	Technology / Version	Use in the Project
Landing	wwwroot/index.html	Public home page, vacancy discovery, navigation and theme toggle.
Register	pages/auth/register.html	Job Seeker and Employer account registration.
Login	pages/auth/login.html	Email/password authentication and role-based redirect.
Job Seeker	pages/jobseeker/dashboard.html	Vacancies, matching, applications, profile/CV and notifications.
Employer	pages/employer/dashboard.html	Vacancies, applicants, rankings, status and recruitment actions.
Administrator	pages/admin/dashboard.html	Administrative dashboard and management views.

4. Backend Technology / Framework

The backend is built with ASP.NET Core Web API on .NET 8. It exposes RESTful controllers and uses dependency injection to connect controllers to application services and repository implementations. JSON is the primary API payload format, and enum values are configured to serialize as readable strings where appropriate.

4.1 ASP.NET Core Web API

- Attribute-routed API controllers using [ApiController], [Route], [HttpGet], [HttpPost], [HttpPut] and [HttpDelete].
- Built-in dependency injection for repositories, services, authentication and matching components.
- HTTPS redirection and standard ASP.NET Core request pipeline.
- Static file hosting for the frontend through UseDefaultFiles and UseStaticFiles.
- Authentication middleware executed before authorization and controller endpoint mapping.
- System.Text.Json serialization with JsonSerializerOptions for readable enum values.

4.2 Layered Application Structure

Category	Technology / Version	Use in the Project
Controllers	HTTP/API boundary	Receives requests, validates

Category	Technology / Version	Use in the Project
		route/user context and returns HTTP responses.
Services	Business logic layer	Implements authentication, profile, vacancy, application, matching and workflow rules.
Repositories	Data access abstraction	Encapsulates EF Core database access and ownership-aware queries.
DTOs	API contracts	Defines request and response models independently from persistence entities.
Entities	Persistence model	Represents relational database entities and navigation relationships.
Mappings	Object mapping	AutoMapper profiles reduce repetitive conversions between models.
Validators	Input validation	FluentValidation rules support structured model validation.
Matching	Domain engine	Contains scoring, ranking, filtering and skill-gap logic.

4.3 Backend Functional Modules

- Authentication and User Management
- Job Seeker Profile and CV Management
- Skills, Experience and Education
- Employer and Company Management
- Vacancy Management
- Matching, Search and Candidate Ranking
- Applications and Application Status
- Contact Requests
- In-App Notifications
- Administrator Dashboard

5. Database Used and Data-Access Technology

The project uses Microsoft SQL Server LocalDB through the MSSQLLocalDB instance for local development. The configured database name is SmartRecruitmentMatchingPlatformDb. Entity Framework Core 8.0.29 provides the object-relational mapping layer between C# entities and SQL Server tables.

Category	Technology / Version	Use in the Project
Database Engine	Microsoft SQL Server LocalDB	Local relational database used by the final project configuration.
LocalDB Instance	(localdb)\MSSQLLocalDB	SQL Server development instance used in the connection string.
Database Name	SmartRecruitmentMatchingPlatformDb	Application database containing recruitment platform data.
ORM	Microsoft.EntityFrameworkCore.SqlServer 8.0.29	SQL Server provider and EF Core data access.
Design-Time Tools	Microsoft.EntityFrameworkCore.Design 8.0.29	Supports migration design-time services.
EF Tooling	Microsoft.EntityFrameworkCore.Tools 8.0.29 / dotnet-ef	Migration listing, database update and EF Core CLI workflows.
Migration Baseline	20260810095554_InitialCreate	Final verified database schema migration in the repository.

5.1 Data Model Coverage

The relational model supports the primary recruitment workflows, including users, refresh tokens, Job Seekers, Job Seeker profiles, CVs, skills, Job Seeker skills, experience, education, employers, companies, vacancies, vacancy skills, applications, contact requests and notifications.

5.2 Why SQL Server and EF Core Fit This Project

- Relational integrity is suitable for users, vacancies, applications and many-to-many skill relationships.
- EF Core provides strongly typed LINQ queries and navigation properties from the C# application layer.
- Migrations provide repeatable schema creation and change tracking.
- SQL Server integrates directly with ASP.NET Core and Visual Studio development workflows.
- Unique constraints and relational keys support important rules such as duplicate-application prevention.

6. Programming Languages Used

Category	Technology / Version	Use in the Project
C#	Primary backend language	Controllers, services, repositories, entities, DTOs, validators, matching engine and tests.
JavaScript	Frontend behavior	API calls, authentication state, forms, dashboards, theme logic and dynamic rendering.
HTML5	Frontend markup	Page structure, forms, tables, dialogs, navigation and role-specific views.
CSS3	Frontend presentation	Responsive layout, visual design, themes, controls, cards, tables and modal styling.
SQL	Relational data operations	Executed primarily through EF Core-generated SQL against SQL Server.
JSON	Data/configuration format	REST payloads, appsettings configuration, JWT settings and global.json SDK configuration.

Frontend framework choice

No React, Angular or Vue framework is required by the final implementation. The frontend intentionally uses HTML, CSS and Vanilla JavaScript to keep the solution simple, portable and directly integrated with the ASP.NET Core host.

7. APIs / Services Used

The platform primarily uses its own internal RESTful Web API. The frontend and Swagger UI communicate with ASP.NET Core controller endpoints over HTTP/HTTPS. The project does not rely on external AI, email/SMS, payment or job-board APIs for its core functionality.

7.1 Internal REST API Services

Category	Technology / Version	Use in the Project
Authentication API	/api/auth/*	Registration, login, refresh token,

Category	Technology / Version	Use in the Project
		password changes and logout.
Job Seeker API	/api/jobseekers/*	Profile, CV, skills, education and experience operations.
Vacancy API	/api/Vacancy/*	Vacancy listing, search and Employer vacancy management.
Matching API	/api/Matching/*	Deterministic score calculation, skill gap and ranked-candidate operations.
Application API	/api/Application/*	Apply, view applications and update application status.
Contact Request API	/api/ContactRequest/*	Employer-to-Job-Seeker contact request workflow and responses.
Notification API	/api/Notifications/*	In-app notification listing, read state and unread count.
Admin API	/api/Admin/*	Administrative dashboard and system-level summary functions.
Skills API	/api/skills/*	Master skill catalogue and skill lookup operations.

7.2 Swagger / OpenAPI Service

Swashbuckle.AspNetCore 6.6.2 generates the OpenAPI specification and Swagger UI. Swagger is configured with a Bearer security definition so authenticated endpoints can be tested interactively by providing a JWT access token.

7.3 Authentication Service

JWT Bearer authentication is implemented with Microsoft.AspNetCore.Authentication.JwtBearer 8.0.29. Token validation checks issuer, audience, lifetime and signing key, and the project configures zero clock skew. Authentication services issue access/refresh tokens and ASP.NET Core authorization enforces role-specific access.

7.4 Services Explicitly Not Used

- No OpenAI, Gemini or other external AI/LLM API is used for matching.
- No machine-learning service is used for candidate scoring.

- No SMTP, SendGrid, SMS gateway or external notification service is required; notifications are in-app.
- No payment gateway is used.
- No external job-board API is integrated.
- No video-interview or assessment service is integrated.

8. Other Tools, Libraries and Technologies

The solution uses several focused NuGet packages and development tools in addition to the .NET platform itself. The following items are present in the finalized project configuration.

8.1 Application NuGet Packages

Category	Technology / Version	Use in the Project
AutoMapper.Extensions.Microsoft.DependencyInjection	8.1.1	Object mapping between domain/entities and DTO-oriented models.
FluentValidation	12.1.1	Declarative validation rules for application input models.
FluentValidation.DependencyInjectionExtensions	12.1.1	Assembly scanning and DI integration for FluentValidation validators.
Mediarq	1.5.1	Referenced application package in the final project configuration.
Microsoft.AspNetCore.Authentication.JwtBearer	8.0.29	JWT Bearer authentication middleware.
Microsoft.EntityFrameworkCore.Design	8.0.29	EF Core design-time services and migration support.
Microsoft.EntityFrameworkCore.SqlServer	8.0.29	Entity Framework Core provider for Microsoft SQL Server.
Microsoft.EntityFrameworkCore.Tools	8.0.29	EF Core tooling support.
Swashbuckle.AspNetCore	6.6.2	Swagger/OpenAPI generation and interactive API documentation.

8.2 Built-In .NET / ASP.NET Core Technologies

- ASP.NET Core Dependency Injection for service and repository registration.

- System.Text.Json for JSON serialization and deserialization.
- JsonStringEnumConverter for readable enum values in API payloads.
- Microsoft.AspNetCore.Identity.PasswordHasher<TUser> for password hashing.
- Microsoft.IdentityModel.Tokens for JWT signing-key and token validation configuration.
- ASP.NET Core Static File Middleware for serving the frontend from wwwroot.
- HTTPS Redirection middleware for secure local/API access.

8.3 Testing Packages

Category	Technology / Version	Use in the Project
xUnit	2.9.3	Primary unit testing framework.
xunit.runner.visualstudio	3.1.4	Visual Studio and test-runner integration for xUnit.
Moq	4.20.72	Mocking repositories/services and isolating units under test.
Microsoft.NET.Test.Sdk	17.14.1	.NET test discovery and execution infrastructure.
coverlet.collector	6.0.4	Code coverage data collection support.
Aspire.Hosting.Testing	9.5.0	Referenced testing support package in the test project.

9. Security and Authentication Stack

Security is implemented using ASP.NET Core authentication and authorization features together with JWT Bearer tokens and hashed passwords. The design separates public operations, such as registration and login, from protected role-specific operations.

Category	Technology / Version	Use in the Project
JWT Bearer Tokens	Access authentication	Signed tokens carry authenticated identity and role information to protected API endpoints.
Refresh Tokens	Session renewal	Stored refresh-token records support access-token renewal and logout/session invalidation workflows.
PasswordHasher<User>	Password security	Hashes user passwords rather than

Category	Technology / Version	Use in the Project
		storing plain-text passwords.
Role Authorization	Access control	Endpoints can require JobSeeker, Employer or Administrator roles.
HTTPS Redirection	Transport security	ASP.NET Core redirects requests to HTTPS in the standard request pipeline.
Protected File Storage	CV protection	CV storage is outside wwwroot, preventing direct public static-file exposure.
JWT Validation	Token verification	Issuer, audience, signing key and token lifetime are validated on protected requests.

Credential handling

A user may register with a real email address as the account identifier, but the Smart Recruitment Platform password is independent of the user's email-provider password. The project does not implement external email delivery or email verification.

10. Matching Engine Technology

The recruitment matching feature is an internal C# rule-based engine. It is implemented through matching-engine, scoring, ranking, filtering and skill-gap components. It reads persisted Job Seeker and vacancy data from the database and calculates a deterministic score that can be explained to users.

Category	Technology / Version	Use in the Project
Skills	50%	Compares Job Seeker skills with required vacancy skills.
Experience	25%	Compares accumulated experience with the vacancy experience requirement.
Education	15%	Compares the Job Seeker education level with the required education level.

Category	Technology / Version	Use in the Project
Location	10%	Compares Job Seeker and vacancy location information.
Salary	0%	Not used as a matching criterion.

10.1 Matching Components

- MatchingEngine - coordinates rule-based scoring.
- MatchScoreCalculator - combines factor scores using configured weights.
- SkillScoreCalculator - evaluates skill compatibility.
- ExperienceScoreCalculator - evaluates experience suitability.
- EducationScoreCalculator - evaluates education suitability.
- LocationScoreCalculator - evaluates location compatibility.
- SkillGapCalculator - returns matched and missing skills.
- CandidateRanker - orders applicants by deterministic match score.
- JobFilter - supports vacancy filtering/search logic.

Important technical classification

This matching implementation is deterministic business logic written in C#. It should be described as rule-based matching, not artificial intelligence or machine learning.

11. Testing and Quality Assurance Stack

Automated tests are maintained in a separate net8.0 test project. The final solution was validated using dotnet build, dotnet test and Entity Framework migration checks. Swagger was also used throughout development to exercise API routes and authenticated workflows.

11.1 Automated Test Areas

- Application service behavior and duplicate application protection.
- Contact request service behavior.
- Deterministic matching-engine score behavior.
- Skill service behavior.
- Notification service behavior.
- Administrator service behavior.

11.2 Final Verification Baseline

Category	Technology / Version	Use in the Project
Build	dotnet build	Final verified result: 0 warnings, 0

Category	Technology / Version	Use in the Project
		errors.
Unit Tests	dotnet test	Final verified result: 12 passed, 0 failed, 0 skipped.
EF Migration Check	dotnet ef migrations list	Final verified migration baseline: 20260810095554_InitialCreate.
API Testing	Swagger UI	Interactive verification of API endpoints, request DTOs and JWT-protected operations.
Browser Smoke Testing	Modern browser	Landing page, auth, dashboards, matching and theme behavior verified during final stabilization.

12. Development, Version Control and Supporting Tools

Category	Technology / Version	Use in the Project
Visual Studio 2022	Primary IDE	Solution editing, debugging, build/test execution and SQL Server Object Explorer support.
.NET CLI	dotnet	Restore, build, run, test and project commands.
EF Core CLI	dotnet-ef	Migration listing and database update commands.
Git	Distributed version control	Branching, commits, merges, status/diff checks and release history.
GitHub	Remote repository hosting	Team collaboration, remote branches and final main-branch source distribution.
Swagger UI	API exploration/testing	Manual endpoint inspection and Bearer-authenticated API testing.
SQL Server LocalDB	Local database runtime	Lightweight local SQL Server database used during development/testing.

Category	Technology / Version	Use in the Project
Browser Developer Environment	Web runtime	Frontend execution, responsive UI validation and HTTP interaction with the local API.

12.1 Source-Control Workflow

Development was organized using module branches, a shared Develop integration branch and a final main branch. The completed final state was pushed so that Develop and main reference the finalized tested implementation.

13. Final Technology Version Matrix

Category	Technology / Version	Use in the Project
.NET Target Framework	net8.0	Runtime target
.NET SDK	8.0.300	global.json baseline
ASP.NET Core Web API	.NET 8 platform	Backend framework
EF Core SQL Server	8.0.29	Database provider
EF Core Design	8.0.29	Design-time support
EF Core Tools	8.0.29	Migration/tooling support
JWT Bearer	8.0.29	Authentication middleware
Swashbuckle.AspNetCore	6.6.2	Swagger/OpenAPI
AutoMapper DI Extensions	8.1.1	Model mapping
FluentValidation	12.1.1	Validation
FluentValidation DI Extensions	12.1.1	Validator registration
Mediarq	1.5.1	Referenced application package
xUnit	2.9.3	Unit testing
Moq	4.20.72	Mocking

Category	Technology / Version	Use in the Project
Microsoft.NET.Test.Sdk	17.14.1	Test execution
coverlet.collector	6.0.4	Coverage support
xUnit VS Runner	3.1.4	Test runner integration
Aspire.Hosting.Testing	9.5.0	Referenced test support

13.1 Technology Stack Summary by Submission Requirement

Category	Technology / Version	Use in the Project
.NET Version used	.NET 8.0, TargetFramework net8.0, SDK 8.0.300	Confirmed
Frontend Technology / Framework	HTML5, CSS3, Vanilla JavaScript, ASP.NET Core static-file hosting	Confirmed
Backend Technology / Framework	ASP.NET Core Web API on .NET 8	Confirmed
Database used	Microsoft SQL Server LocalDB with Entity Framework Core	Confirmed
Programming Languages	C#, JavaScript, HTML, CSS, SQL/JSON supporting formats	Confirmed
APIs / Services used	Internal REST API, Swagger/OpenAPI, JWT Auth, in-app notification/contact services	Confirmed
Other Tools / Libraries / Technologies	EF Core, AutoMapper, FluentValidation, Swashbuckle, xUnit, Moq, Git, GitHub, Visual Studio	Confirmed

14. Conclusion

The Smart Recruitment Matching Platform uses a modern but intentionally straightforward full-stack architecture. .NET 8 and ASP.NET Core Web API provide the backend foundation, SQL Server LocalDB and Entity Framework Core provide relational persistence, and HTML5/CSS3/Vanilla JavaScript provide a responsive browser interface. JWT authentication and role-based authorization secure user-specific workflows, while Swagger/OpenAPI supports API documentation

and testing. The recruitment matching capability is implemented as deterministic C# business logic using skills, experience, education and location rather than external AI services.

This stack is well aligned with the project requirements because it provides secure authentication, structured data storage, testable REST APIs, clear separation of concerns, responsive web access, explainable matching logic and a development workflow that can be built, tested and maintained using standard Microsoft and open-source .NET tooling.

Final technical status

The final project baseline is on .NET 8, builds successfully with 0 warnings and 0 errors, passes all 12 automated tests, and retains a single verified InitialCreate database migration.

Technical Evidence Basis

This document was prepared from the finalized main-branch project configuration and source structure, including global.json, the main application .csproj, Program.cs, appsettings.json, the test-project .csproj, migration state and final verification results.